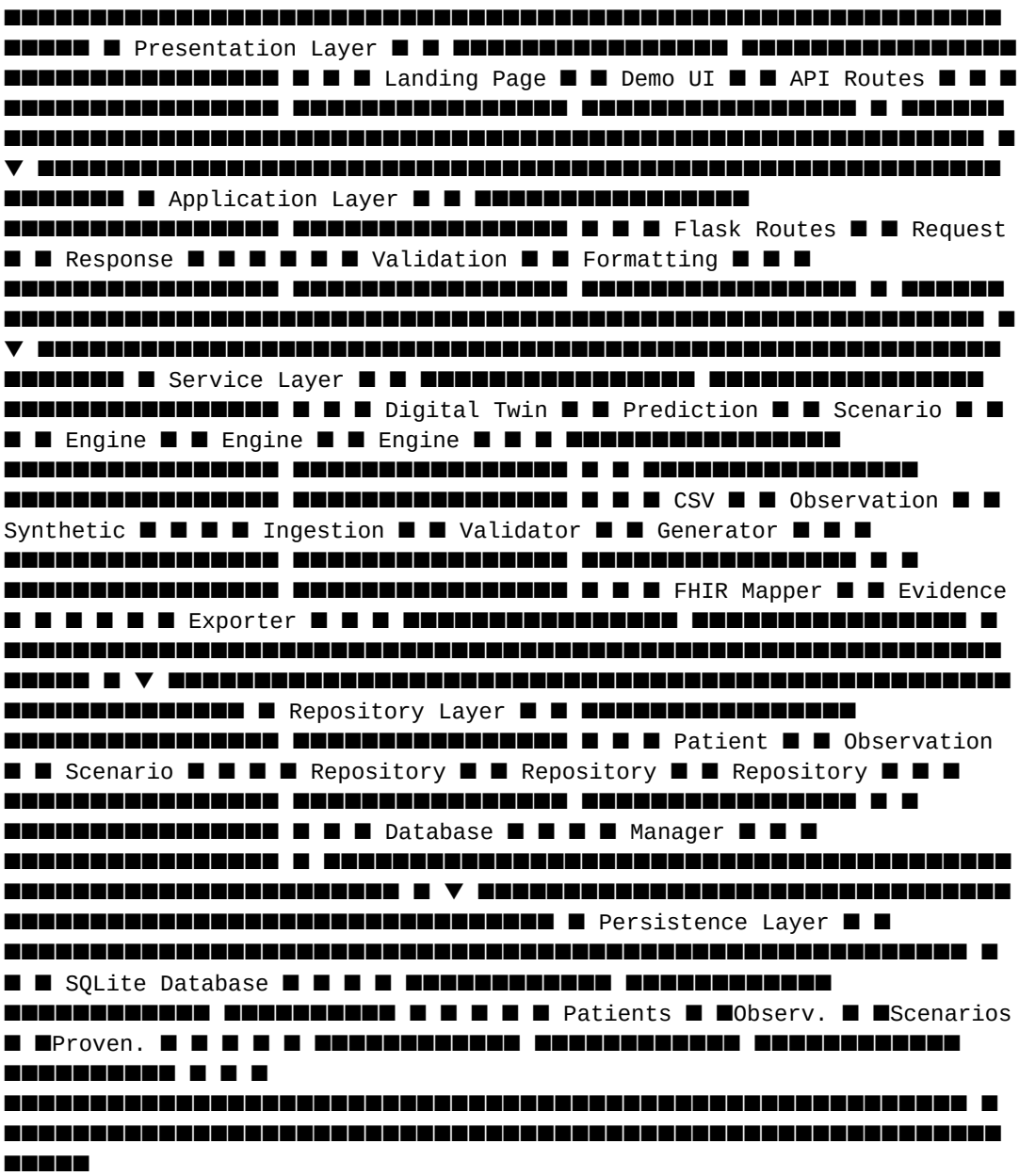
Swasthya Twin - System Architecture

System Overview

Swasthya Twin is a modular, evidence-oriented Digital Twin system built with Flask, SQLite, and Python standard libraries. The architecture separates concerns into distinct layers for maintainability and testability.

High-Level Architecture



Component Descriptions

Presentation Layer

• Landing Page (templates/index.html): Project overview, capabilities, architecture, disclaimers

• Demo UI (templates/demo.html): Patient management, observation entry, twin state, risk assessment, scenarios

• API Routes (app.py): RESTful endpoints, health check, JSON handling

Application Layer

• Flask Application (app.py): WSGI entry point, route definitions, request handling, security, error handling

Service Layer

• Digital Twin Engine: Computes baselines, rolling averages, trends, data quality

• Prediction Engine: Rules-based risk assessment, feature contributions, confidence scoring

• Scenario Engine: Hypothetical scenarios, modification validation, baseline comparison

• CSV Ingestion Service: CSV parsing, validation, batch processing, duplicate detection

• Observation Validator: Range, unit, timestamp validation, provenance tracking

• Synthetic Data Generator: Reproducible patient generation, fixed-seed determinism

• FHIR Mapper: Domain to FHIR resource mapping, bundle generation

• Evidence Exporter: JSON bundles, CSV export, summary reports

Repository Layer

• Patient Repository: CRUD operations, ID queries, synthetic filtering

• Observation Repository: CRUD, patient queries, type filtering, batch operations

• Scenario Repository: CRUD, patient queries, modification persistence

• Database Manager: SQLite connections, schema initialization, indexes, backup

Persistence Layer

• SQLite Database: Patients, Observations, Scenarios, Provenance tables with constraints and indexes

Data Flow

Observation Ingestion Flow

User Input/CSV Upload ■ ▼ Validation ■ ▼ Provenance Recording ■ ▼ Database Storage ■ ▼ Digital Twin State Update ■ ▼ Risk Assessment Update

Scenario Simulation Flow

User Creates Scenario ■ ▼ Modification Validation ■ ▼ Load Baseline State ■ ▼ Apply Modifications ■ ▼ Compute New Risk ■ ▼ Compare with Baseline ■ ▼ Return Scenario Results

Risk Assessment Flow

Load Patient Observations ■ ▼ Compute Digital Twin State ■ ▼ Check Data Quality ■ ▼ Compute Feature Contributions ■ ▼ Aggregate Risk Score ■ ▼ Determine Risk Level ■ ▼ Generate Explanations

Security Considerations

Input Validation: All user inputs validated at service layer, range checking, unit validation, timestamp validation

SQL Injection Prevention: Parameterized queries throughout, no string concatenation, foreign key constraints

File Upload Security: File type restrictions (CSV only), 16 MB size limits, secure filename handling, upload isolation

Session Security: CSRF protection, secure session configuration, environment-based secret keys

Data Privacy: No external API calls, local-only storage, no credentials in logs, synthetic data for demo

Technology Stack

Component	Technology
Backend	Python 3.14.6, Flask 3.0.3, SQLite
Frontend	HTML5, CSS3, Vanilla JavaScript, Jinja2
Dependencies	NumPy 1.26.4, Pandas 2.2.2, Gunicorn 21.2.0
Development	Pytest 8.2.0, pytest-cov 5.0.0

Deployment Architecture

Production deployment on Render: Internet → Render Load Balancer → Gunicorn WSGI Server → Flask Application → SQLite Database (Ephemeral Storage)

• Stateless application design

• Ephemeral filesystem (data resets on redeploy)

• Health check endpoint for monitoring

• Memory-efficient (<512 MB target)

• No background workers

• No external dependencies